

Makefile

Para poder utilizar o Makefile antes precisa fazer a instalação do MinGW para o windows e o CMake para linux, conforme as instruções apresentadas nos slides passados na primeira aula de laboratório.

1º modelo:

compile: #todos os arquivos .c

gcc -o bin/main #todos os arquivos .c -g -Wall

Para compilar o projeto, certifique-se de estar no diretório raiz do projeto (onde o arquivo Makefile está localizado) e de que o diretório "bin" já existe(pasta onde ficará o executável). No terminal, execute o seguinte comando:

make compile

./bin/main

Termos:

- **compile**: No contexto do Makefile, isso é chamado de **alvo** (ou *target*). É o nome da ação que você está criando. Quando você digita o comando para rodar o Makefile, o sistema procura por esse nome para saber quais arquivos verificar e qual comando executar logo abaixo dele.
- **-g**: É uma *flag* (opção) passada para o compilador gcc. Ela instrui o compilador a incluir **informações de depuração (debug)** no arquivo executável final. Isso é essencial se você precisar usar ferramentas como o gdb ou o valgrind para rastrear erros e falhas de segmentação (*segmentation faults*) no seu código.
- **-Wall**: Significa "*Warnings all*". Esta *flag* ativa a maioria das mensagens de aviso do compilador. Ela analisa o seu código em busca de construções suspeitas, variáveis declaradas que não foram usadas, tipos incompatíveis ou lógicas que, embora compilem, podem causar erros durante a execução.

2º modelo:

Compilador e flags

CC = gcc

CFLAGS = -Wall -linclude

```

# Diretórios
SRC_DIR = src
OBJ_DIR = $(SRC_DIR)
BIN = programa

# Arquivos fonte e objetos
SRCS = $(SRC_DIR)/main.c $(SRC_DIR)/seus arquivos todos precedidos de
$(SRC_DIR)/
OBJS = $(SRCS:.c=.o)

# Regra padrão
all: $(BIN)

# Linkagem final
$(BIN): $(OBJS)
    $(CC) $(CFLAGS) -o $(BIN) $(OBJS)

# Compilação dos .c
$(OBJ_DIR)/%.o: $(SRC_DIR)/%.c
    $(CC) $(CFLAGS) -c $< -o $@

# Executar com arquivo de teste
run: $(BIN)
    ./$(BIN) testes/arquivo.txt(seus testes)

# Limpar arquivos compilados
clean:
    rm -f $(OBJ_DIR)/*.o $(BIN)

```

Configurações iniciais: Define o compilador como gcc, ativa os alertas (-Wall), inclui a pasta de cabeçalhos (-Iinclude) e define que o programa final vai se chamar **programa**.

Regras de compilação (all, \$(BIN) e %.o): Ele pega os seus códigos .c da pasta src (main.c,...), converte cada um em arquivos intermediários .o e depois junta todos eles para criar o executável. Ao digitar apenas **make** no terminal, esse processo acontece automaticamente.

Comando de execução (run): Facilita os testes. Ao digitar **make run** no terminal, ele executa o programa já passando o arquivo testes/mapa1.txt como entrada.

Comando de limpeza (clean): Ao digitar **make clean** no terminal, ele apaga o arquivo executável e todos os arquivos .o g

terminal:

make

make run